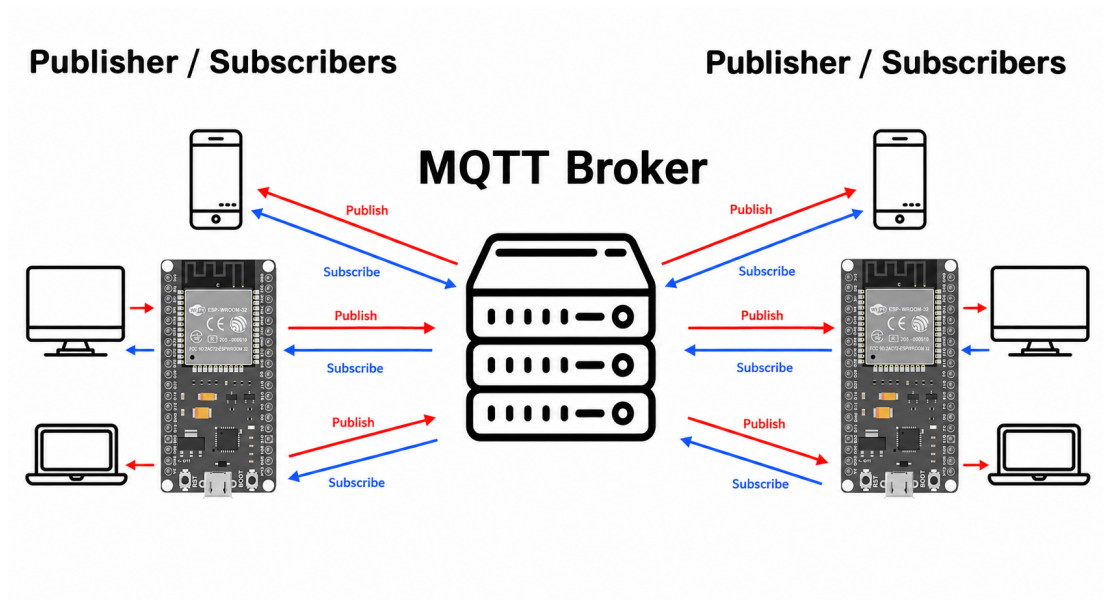


Two-Way Wireless Interlock System

Dual-ESP32 IR-to-Motor Safety Link over MQTT



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents the design and implementation of a two-way wireless safety interlock built from a pair of ESP32 boards communicating through MQTT, over a shared, internet-reachable public MQTT broker. Each board is simultaneously a publisher and a subscriber: it reads its own local IR obstacle sensor, publishes that state as a JSON message to a topic that belongs to it, and at the same time subscribes to its partner's topic to receive the partner's state, which is what actually gates its own local DC motor. This is a deliberate re-implementation of the same interlock concept previously built over ESP-NOW (Task 2.4), replacing the direct peer-to-peer radio link with a broker-mediated publish/subscribe link, so the two boards no longer need to be paired by MAC address or even be on the same WiFi network – they only need reachability to the same broker.

The core safety property of the system carries over unchanged: the fail-safe link timeout. If a board stops receiving fresh messages from its partner's topic for more than two seconds – because the partner lost power, lost its WiFi connection, the broker connection dropped, or any other point of the chain failed – it unconditionally stops its own motor rather than continuing to run on the last command it happened to receive. Losing the link, at any layer, degrades to a safe, known mechanical state instead of an undefined one.

Four status LEDs per board (transmit heartbeat, motor state, IR/obstacle state, and receive/link state) make the system's internal state observable from the outside without needing a Serial monitor.

2. System Overview & Goal

Two interns, each with one ESP32 board, one IR obstacle sensor, and one DC motor with driver, form a cross-wired safety loop, now mediated by a shared broker instead of a direct radio link:

```
[Intern A ESP32] --- (IR Sensor) --> publish --> [public MQTT broker] --> subscribe --> (Motor) [Intern
                                     B ESP32]
                                     tern A ESP32
    <-- subscribe <-- [public MQTT broker] <-- publish <-- (IR Sensor) [Intern B ESP32] --- (Motor)
```

- Intern A's IR sensor state decides whether *Intern B's* motor is allowed to run.
- Intern B's IR sensor state decides whether *Intern A's* motor is allowed to run.
- Both boards run the exact same firmware logic (this report's `main.cpp`); the only per-board difference is the pair of identity constants `MY_ID` and `PARTNER_ID` in `config.h`, which determine which topic each board publishes to and which topic it subscribes to.

The "your sensor gates my motor" arrangement is unchanged from the ESP-NOW version: neither motor can be commanded to run by its own local sensor – only by its remote partner's sensor, delivered over a link that is itself continuously safety-checked (Section).

3. Hardware Architecture & Pin Mapping

Each of the two boards is wired identically. Pin assignments come directly from `config.h`, so there is a single place to retune the wiring without touching the application logic elsewhere.

Peripheral	Signal	ESP32 GPIO	Role
IR obstacle sensor	Digital out	GPIO 34 (IR_PIN)	Input-only ADC pin; reads local obstacle state
Motor driver (H-bridge)	IN1	GPIO 26 (MOTOR_IN1)	Direction/enable line 1
Motor driver (H-bridge)	IN2	GPIO 27 (MOTOR_IN2)	Direction/enable line 2 (held LOW)
Motor driver (H-bridge)	ENA	GPIO 25 (MOTOR_ENA)	PWM speed-enable line, driven once at full duty
Status LED	Transmit heartbeat	GPIO 16 (TX_LED)	Toggles on every successful state publish
Status LED	Motor state	GPIO 19 (MOTOR_LED)	Mirrors <code>motorShouldRun</code>
Status LED	IR/obstacle state	GPIO 18 (IR_LED)	Lit when an obstacle is currently detected
Status LED	Link/receive state	GPIO 17 (RX_LED)	Lit while the partner link is alive

Table 1: ESP32 pin mapping, identical on both interns' boards.

Network prerequisites

Unlike the ESP-NOW version, both boards need an actual WiFi network with internet (or LAN) reachability to the broker; there is no requirement that the two boards share the same access point, only that both can reach `MQTT_BROKER` on `MQTT_PORT`.

4. Schematic and Circuit

Wiring Schematic

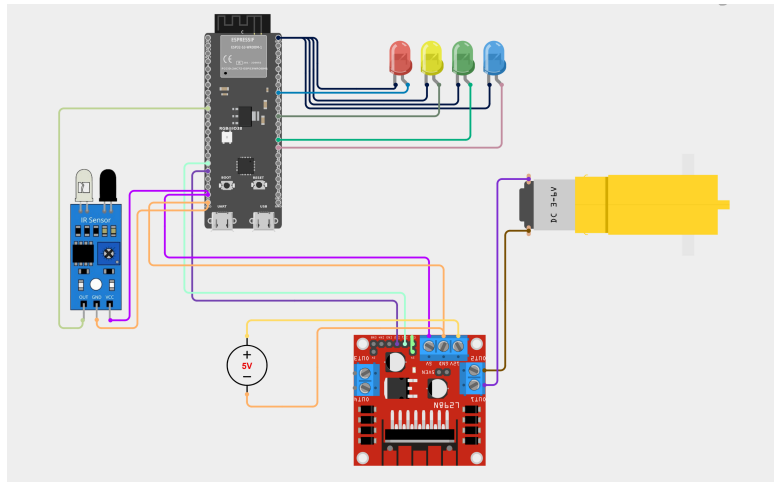


Figure 1: wiring schematic

Assembled Circuit

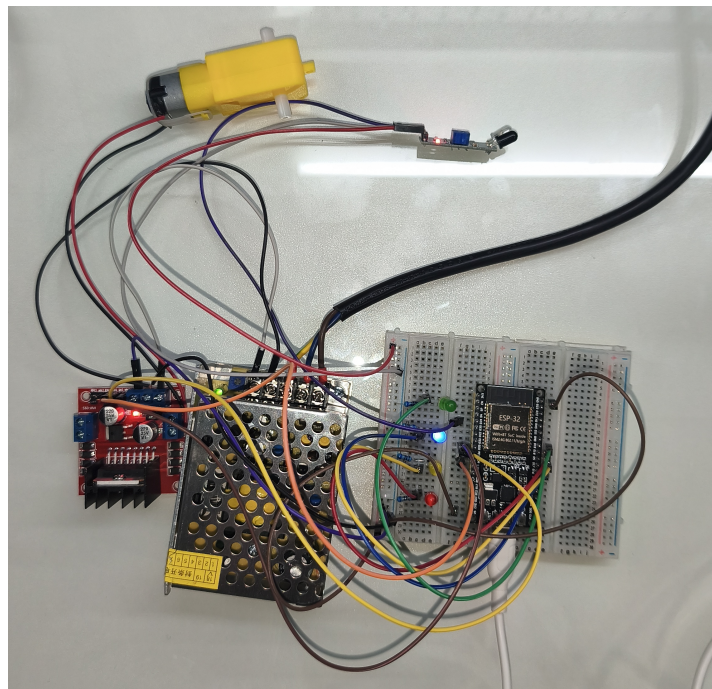


Figure 2: Photo of the assembled circuit

5. Software Architecture

The firmware is split into small, single-responsibility classes, each in its own header, plus one implementation file for the networking layer:

File	Responsibility
<code>config.h</code>	Every credential, GPIO pin, topic constant, and timing constant used by the sketch. No other file contains a raw pin number, topic string, or broker address.
<code>InterlockMessage.h</code>	The single wire-format message exchanged between the two boards, plus its JSON encode/decode logic.
<code>IRSensor.h</code>	Reads and debounces the local IR obstacle sensor.
<code>MotorController.h</code>	Drives the H-bridge on/off.
<code>StatusIndicator.h</code>	Owens the four status LEDs.
<code>LinkMonitor.h</code>	Tracks the time since the last message actually arrived from the partner and derives the fail-safe <code>isLinkLost()</code> flag.
<code>MqttInterlockClient.h/.cpp</code>	Owens the WiFi connection, the broker connection, subscription/publish calls, and the incoming-message callback.
<code>main.cpp</code>	Wires all of the above together in <code>setup()/loop()</code> : reads the sensor, evaluates the fail-safe, drives the motor and LEDs, and publishes on a fixed cadence.

Non-blocking main loop

`loop()` has no blocking `delay()` beyond a short, fixed 20 ms pacing delay at the very end of each iteration. WiFi/broker reconnection, the IR debounce timer, the link-timeout check, and the periodic publish are all evaluated every pass against `millis()`-based timestamps, so the fail-safe check (Section) is re-evaluated on essentially every iteration.

6. MQTT Communication Protocol

Broker connection and reconnection

`MqttInterlockClient` layers two independent, non-blocking reconnection routines, both driven from `loop()`: one for the WiFi link, one for the broker session. Each is only attempted when the underlying layer is actually down, and broker reconnect attempts are rate-limited by `MQTT_RECONNECT_MS` so a broker outage does not flood reconnect attempts.

```

1 void MqttInterlockClient::loop() {
2   if (WiFi.status() != WL_CONNECTED) {
3     connectWiFi();
4   }
5   if (!_mqttClient.connected()) {
6     connectBroker();
7   }
8   _mqttClient.loop();
9 }

```

Listing 1: `MqttInterlockClient.cpp` – `loop()`

The broker's address and port are read from two constants, `MQTT_BROKER` and `MQTT_PORT`, defined once in `config.h`. No other file in the project contains a broker hostname. Switching to a different public MQTT broker – for example moving away from `test.mosquitto.org` – is therefore a one-line edit in `config.h`; nothing else in the firmware needs to change.

```
1 #define MQTT_BROKER      "test.mosquitto.org"
2 #define MQTT_PORT        1883
3 #define MQTT_USERNAME    ""
4 #define MQTT_PASSWORD    ""
```

Listing 2: `config.h` – broker identity (excerpt)

Each board builds a unique MQTT client ID at connect time from its own ID and its chip's efuse MAC, so both boards can connect to the same broker without a client-ID collision:

```
1 String clientId = String("esp32-") + MY_ID + "-" + String((uint32_t)ESP.getEfuseMac(),
    HEX);
2
3 bool connected;
4 if (strlen(MQTT_USERNAME) > 0) {
5     connected = _mqttClient.connect(clientId.c_str(), MQTT_USERNAME, MQTT_PASSWORD);
6 } else {
7     connected = _mqttClient.connect(clientId.c_str());
8 }
```

Listing 3: `MqttInterlockClient.cpp` – `connectBroker()` (excerpt)

Topic structure

Every topic is built from a single shared prefix plus a per-board ID, so each board publishes to its own topic and subscribes only to its partner's:

```
1 #define MY_ID             "studentB"
2 #define PARTNER_ID        "studentA"
3
4 #define TOPIC_PREFIX      "iot_field_training/task2_4/interlock/"
5 #define MY_TOPIC          TOPIC_PREFIX MY_ID
6 #define PARTNER_TOPIC     TOPIC_PREFIX PARTNER_ID
```

Listing 4: `config.h` – topics

A board never subscribes to its own topic, so it never receives an echo of its own publish. The only per-board difference in the entire codebase is the pair `MY_ID` / `PARTNER_ID`; the second intern's copy of `config.h` simply swaps the two values.

Message format

The payload is a small JSON object, encoded and decoded through `ArduinoJson`, rather than a raw binary struct copied over the air:

```
1 struct InterlockMessage {
2     String    senderID;
3     bool      runMotor;
4     uint32_t  timestamp;
5
6     String toJson() const {
7         JsonDocument doc;
8         doc["senderID"] = senderID;
9         doc["runMotor"] = runMotor;
```

```

10     doc["timestamp"] = timestamp;
11     String out;
12     serializeJson(doc, out);
13     return out;
14 }
15
16 static bool fromJson(const uint8_t* payload, unsigned int length, InterlockMessage&
    out) {
17     JsonDocument doc;
18     DeserializationError err = deserializeJson(doc, payload, length);
19     if (err) { return false; }
20     out.senderID = doc["senderID"] | "";
21     out.runMotor = doc["runMotor"] | false;
22     out.timestamp = doc["timestamp"] | 0;
23     return true;
24 }
25 };

```

Listing 5: InterlockMessage.h – the wire message

A published message looks like:

```

1 {"senderID":"studentB","runMotor":true,"timestamp":184532}

```

Listing 6: Example payload on MY_TOPIC

Choosing a text/JSON payload over a raw struct trades a few bytes of wire overhead for a message that is human-readable in any MQTT inspector, independent of struct packing or endianness on either board.

Publish cadence

Every PUBLISH_INTERVAL_MS (200 ms), main.cpp publishes the board's own current, debounced IR state to its own topic:

```

1 if (millis() - lastPublish > PUBLISH_INTERVAL_MS) {
2     lastPublish = millis();
3     mqttClient.publishState(irSensor.stableRawState());
4     statusLeds.toggleHeartbeat();
5 }

```

Listing 7: main.cpp – periodic publish

As with the ESP-NOW version, publishing continuously – rather than only on a state change – is what makes the fail-safe timeout possible: the receiving side is watching for the *absence* of a recent message, not for an explicit "stop" message, so the sender must keep publishing even while its own IR state is unchanged.

Receiving

Incoming messages arrive through a single PubSubClient callback, registered once in begin() and dispatched to the owning instance via a static trampoline:

```

1 void MqttInterlockClient::handleMessage(char* topic, uint8_t* payload, unsigned int
    length) {
2     InterlockMessage incoming;
3     if (InterlockMessage::fromJson(payload, length, incoming)) {
4         _receivedRunMotor = incoming.runMotor;
5         _linkMonitor.notifyPacketReceived();
6     }

```

```
7 }
```

Listing 8: MqttInterlockClient.cpp – handleMessage()

Only a message that parses successfully as valid JSON updates `_receivedRunMotor` and notifies the `LinkMonitor`; a malformed payload is silently dropped rather than being treated as a fresh, trustworthy packet.

7. IR Sensor Debouncing

Raw digital IR breakout modules commonly chatter for a short window around the exact obstacle-detection threshold. Rather than trusting the instantaneous `digitalRead()` value directly, `IRSensor` runs it through a settle-time debounce before it is ever published or used to drive the local IR LED:

```
1 void update() {
2     bool current = digitalRead(_pin);
3     if (current != _prevReading) {
4         _prevReading = current;
5         _lastChanged = millis();
6     }
7     if (millis() - _lastChanged > _debounceMs) {
8         _stableState = _prevReading;
9     }
10 }
```

Listing 9: IRSensor.h – update()

Every time the raw reading flips, the settle timer restarts; `_stableState` is only updated once the raw reading has held steady for a continuous `_debounceMs` (500ms). A single noisy edge at the detection boundary cannot, by itself, toggle the partner’s motor – the value has to be genuinely stable before it is trusted and published.

8. Fail-Safe Link Timeout

The central safety requirement is unchanged from the ESP-NOW version: losing the link must always result in the local motor stopping, never in the motor continuing to run on stale data. Here “the link” spans WiFi, the broker connection, and the partner’s own health, and a single `LinkMonitor` collapses all of that into one boolean:

```
1 bool isLinkLost() const {
2     return (millis() - _lastArrival) > _timeoutMs;
3 }
```

Listing 10: LinkMonitor.h

```
1 bool linkLost      = linkMonitor.isLinkLost();
2 bool motorShouldRun = (!linkLost) && mqttClient.receivedRunMotor();
```

```
3 motor.setRunning(motorShouldRun);
```

Listing 11: main.cpp – fail-safe evaluation

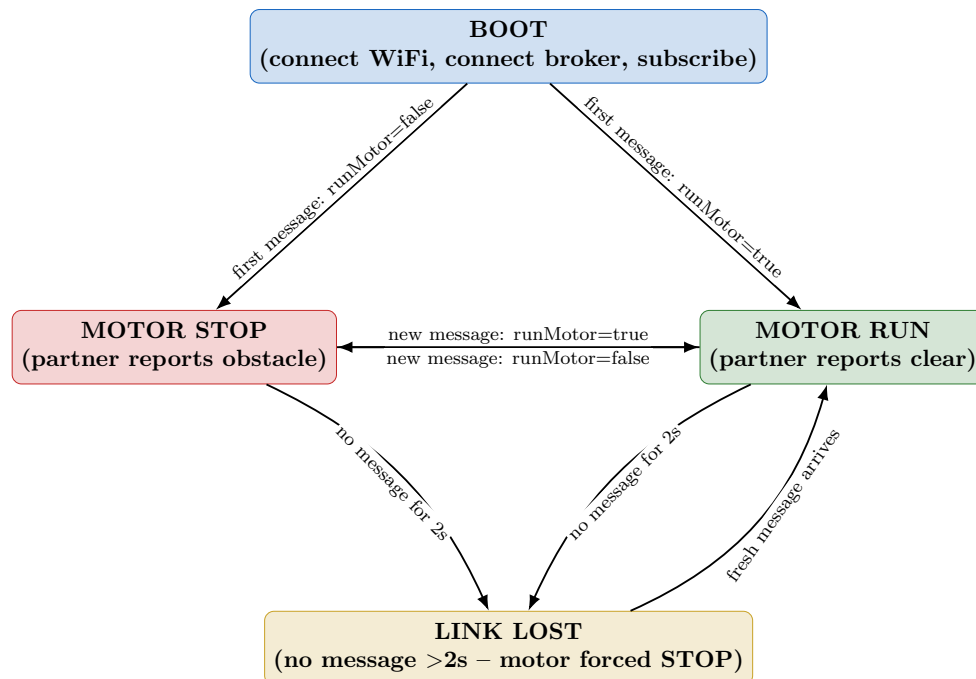
`_lastArrival` is only ever advanced from `notifyPacketReceived()`, called exclusively from inside `handleMessage()` on a successfully parsed packet from the partner's topic. If more than `LINK_TIMEOUT_MS` (2000 ms) elapse without one – because the partner lost power, its WiFi dropped, its broker session dropped, or the broker itself became unreachable – `linkLost` becomes true and `motorShouldRun` is forced false regardless of whatever `receivedRunMotor` last held.

Why the timeout is measured on arrival, not on content

The timeout does not care *what* the last received value was – only *when* it arrived. If the partner loses power mid-session, no further messages of any kind will arrive on its topic, including an explicit stop message. Tying the fail-safe to elapsed silence rather than to message content is what makes the interlock safe even against a partner – or a broker – that fails completely, not only against one that is still alive and well-behaved.

9. System State Diagram

The diagram below shows one board's motor-control state machine, driven by the debounced value last received *from the partner's topic* and by the link's liveness as evaluated by `LinkMonitor`.



Reading the diagram

`LINK LOST` is reachable from either running state and always forces the motor off; leaving it lands directly back in `MOTOR RUN` or `MOTOR STOP` according to whatever the first fresh message actually

contains, rather than through any intermediate "reconnecting" state, since `handleMessage()` always updates `_receivedRunMotor` and notifies the `LinkMonitor` together, on every valid packet.

10. Status LED Indicators

LED	Meaning
Transmit heartbeat (GPIO 16)	Toggles every time <code>publishState()</code> runs, roughly every <code>PUBLISH_INTERVAL_MS</code> ; a steadily blinking LED confirms the board is alive and actively publishing, independent of whether the partner ever answers
Motor state (GPIO 19)	Directly mirrors <code>motorShouldRun</code> – lit exactly when the local motor is being driven
IR/obstacle state (GPIO 18)	Lit when this board's own debounced IR sensor currently reports an obstacle
Link/receive state (GPIO 17)	Lit while <code>!linkLost</code> – goes dark the instant the 2-second fail-safe window expires, giving an immediate visual cue of link loss even without Serial

11. Design Notes – Efficient Choices

1. **Broker as rendezvous instead of a hardcoded peer.** Replacing the ESP-NOW peer MAC address with a pair of topics on a shared broker removes the need for either board to know the other's radio address, and lets the two boards sit on different WiFi networks entirely, as long as both can reach the broker.
2. **Single point of broker configuration.** `MQTT_BROKER` and `MQTT_PORT` are the only place a broker hostname appears in the firmware; every class that needs to talk to the broker reads them from `config.h` rather than holding its own copy, so trying a different public MQTT broker is a one-line change.
3. **Layered, independent reconnection.** WiFi reconnection and broker reconnection are handled as two separate, non-blocking checks in `loop()` (Section 6), so a broker-only outage does not require re-joining WiFi, and a WiFi drop is retried without waiting on the broker layer.
4. **Silence-based fail-safe instead of a content-based one.** Keying the timeout off elapsed time since the last received message (Section), rather than off message content, protects against a partner – or a broker path – that fails completely, including cases where no explicit stop can ever be sent.
5. **Debounce before publish, not after receipt.** The settle filter (Section 7) is applied to the outgoing value before it is ever published, so both boards react to the same, already-clean signal.

6. **JSON payload via a single shared struct.** Because both boards compile the identical `InterlockMessage.h`, there is no possibility of the two boards disagreeing on field names or types, and the payload remains readable in any generic MQTT client used to debug the link.
7. **Non-blocking loop.** With no blocking `delay()` beyond the fixed 20 ms pacing tick, the fail-safe check, the IR debounce timer, the reconnection routines, and the periodic publish are all re-evaluated on essentially every iteration.
8. **Visual state without Serial.** All four LEDs together let a person standing in front of either board diagnose "is it alive," "does it currently see an obstacle," "is the link up," and "is the motor on" at a glance.

12. Known Limitations & Considerations

- **Public, unauthenticated broker.** With `MQTT_USERNAME` and `MQTT_PASSWORD` left empty, the connection is anonymous, and the topic namespace is not access-controlled – anyone else connected to the same public MQTT broker who knows or guesses the topic prefix could publish spoofed messages or read the interlock's traffic.
- **Unencrypted transport.** The connection uses plain `MQTT_PORT` (1883), not TLS, so payloads and credentials (if any were set) travel in clear text over the network.
- **No QoS or delivery guarantee beyond the fail-safe.** `PubSubClient` defaults to QoS 0 for both publish and subscribe, so an individual dropped message is not retried – it is implicitly tolerated by the 2-second timeout, which is appropriate here since messages are published continuously.
- **No sender validation.** `handleMessage()` accepts any well-formed JSON delivered on the subscribed topic; it does not check that `senderID` matches the expected partner ID before trusting the payload.
- **One-directional motor drive.** `MOTOR_IN2` is always held LOW; only `MOTOR_IN1` toggles, so the driver is used purely as an on/off switch for a single direction.
- **Hardcoded WiFi and broker credentials.** `WIFI_SSID`, `WIFI_PASSWORD`, and the broker address all live as plain `#define` constants in `config.h`, committed with the rest of the source.

13. Results & Verification

Serial verification

With `Serial.begin(115200)` active, each loop iteration prints the current `linkLost`, `receivedRunMotor`, and `motorShouldRun` values, used during bring-up to confirm the fail-safe logic transitioned correctly as the partner board was power-cycled and as WiFi/broker connectivity was interrupted.

```
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
```

Figure 3: Serial monitor showing linkLost / receivedRunMotor / motorShouldRun during a live obstacle test

Interlock verification

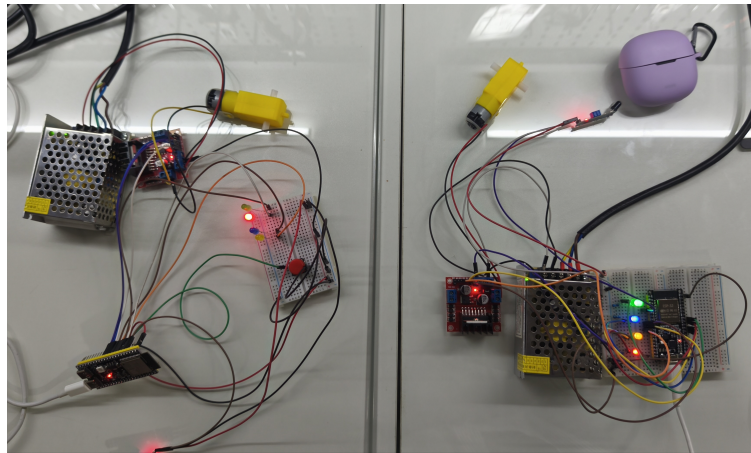


Figure 4: Intern A blocks their IR sensor; Intern B's motor stops within one debounce window

Fail-safe verification

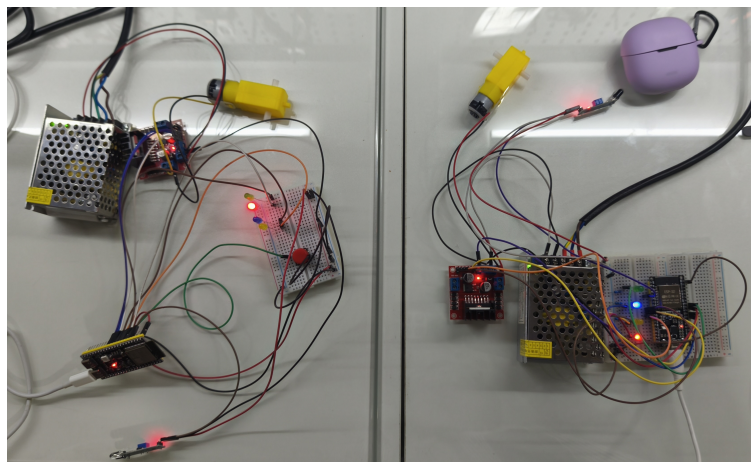


Figure 5: Intern B's board is powered off; Intern A's motor stops within 2 seconds and the Link LED goes dark

14. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	2	One per intern; identical firmware, differing only in MY_ID / PARTNER_ID
IR obstacle sensor module	2	Digital output, one per board, wired to GPIO 34
DC motor + H-bridge driver module	2	Driven via IN1/IN2/ENA per board
Status LEDs (+ current-limiting resistors)	8	4 per board (transmit, motor, IR, link)
Breadboards + jumper wires	2 sets	–
USB cables	2	Programming and power, one per board
WiFi network with internet reachability	–	Required for both boards to reach the public MQTT broker

15. Future Improvements

- **TLS transport** on the broker's secure port, to stop payloads and credentials from traveling in clear text.
- **Authenticated, access-controlled topics**, using MQTT_USERNAME/MQTT_PASSWORD together with a broker ACL scoped to the pair's topic prefix.
- **Sender-ID validation** in handleMessage(), so a message is only trusted if senderID matches the expected PARTNER_ID.
- **Last Will and Testament** on the broker connection, so a board's disconnect is signaled immediately rather than only being inferred after the 2-second silence window.
- **QoS 1** on both publish and subscribe, for broker-acknowledged delivery instead of best-effort QoS 0.
- **Bidirectional motor control** using both MOTOR_IN1 and MOTOR_IN2.